



Faculty of Engineering: Ain Shams University.



AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
International Credit Hour Engineering Programs (ICHEP)
Computer Engineering and Software Systems Program (CESS)

CSE361
Computer Networking
Tiny Telemetry v1

Team members:	Youssef Moustafa Elsayed	22P0047
	Mostafa Fouad	22P0077
	Nour Eldeen Ahmed Rehab	22P0040
	Abdelrahman Mostafa	22P0053
	Mohamed Yasser Khallaf	22P0245
	Ibrahim Shaker Hammad	22P0056



Table of Contents

1. Introduction3

2. Protocol Architecture Overview.....3

3. Methodology: Experimental Automation.....3

4. Experimental Results and Validation4

5. Limitations8

6. Conclusion8



1. Introduction

This report documents the design and experimental evaluation of the IoT Telemetry Protocol (ITP), a custom transport layer designed for sensor devices that periodically transmit temperature and humidity data over UDP¹. The protocol prioritizes simplicity and low overhead for embedded nodes while implementing a "Hybrid Reliability" model. This model provides best-effort delivery for standard telemetry streams but enforces reliable, acknowledged delivery for critical "Priority" events.

2. Protocol Architecture Overview

The ITP operates on a client-server model over UDP (default port 5000).

- **Client:** Operates as a state machine with two transmission modes: *Normal Mode* (sending every 1.0s) and *Batch Mode* (buffering $N=5$ readings).
- **Server:** Acts as a stateful coordinator. It maintains distinct state vectors for each device to track sequence progression, reorder packets via a jitter buffer, and manage session lifecycles.
- **Message Format:** Uses a 12-byte fixed header (containing DeviceID, SeqNum, Timestamp, Flags) and a variable payload, ensuring modularity.

3. Methodology: Experimental Automation

To ensure reproducible verification of the protocol's reliability mechanisms, a custom Python test suite (test.py) was developed. This framework integrates three key components:

1. **Traffic Shaping (Netem):** Injects Linux kernel-level impairments (packet loss, delay, jitter).
2. **Process Management:** Orchestrates the lifecycle of the Server and multiple Client instances⁹.
3. **Packet Capture (Scapy):** Sniffs UDP traffic for post-analysis in Wireshar.

Five specific scenarios were executed to stress-test the protocol's design.



4. Experimental Results and Validation

This section details the results of the automated tests. It includes quantitative metrics and qualitative "Proofs" (log analysis) demonstrating the protocol's behavior under stress.

4.1. Test Scenario 1: 5% Packet Loss

Objective: Verify that the server detects gaps in standard streams while the client successfully retransmits critical Priority packets.

Quantitative Metrics:

- **Packets Received:** 65
- **Sequence Gap Count:** 2

Validation Analysis:

- **Proof 1: Priority Retry Logic (Reliability)**
 - **Observation:** Packet #23 (Priority) was lost. The client entered a blocking wait state.
 - **Evidence:** The client log recorded an ACK_TIMEOUT, followed by ACK_RETRY wait=1s, and finally a successful ACK_RECEIVED after the second attempt. The server log confirmed handling the retry safely: PRIORITY_DUPLICATE ACK_RESENT.
 - **Conclusion:** Critical data is guaranteed delivery even when the network drops packets.

1	1	0	23	1.77E+09	2	2797	19.8	45.1	1.77E+09
---	---	---	----	----------	---	------	------	------	----------

- **Proof 2: Gap Detection (Best-Effort)**
 - **Observation:** Standard Packet #10 was lost.
 - **Evidence:** The server received Sequence 9 and then Sequence 11. It immediately calculated SEQ_GAP expected=10 got=11 missing=1.
 - **Conclusion:** The protocol correctly accepted the loss of non-critical data but logged the event for Packet Loss Rate (PLR) monitoring.

1	1	0	9	1.77E+09	2	10231	19.3	44.6	1.77E+09
1	1	0	11	1.77E+09	0	8951	19.5	44.6	1.77E+09

- **Proof 3: Heartbeat Liveness**
 - **Evidence:** Despite the lossy network, HEARTBEAT_RECEIVED logs appeared every 5 seconds.
 - **Conclusion:** Session visibility is maintained independent of the data stream.

Test Name	packets_received	bytes_per_report	duplicate_rate	sequence_gap_count	cpu_ms_per_report
Packet Loss 5%	65	16	0.0152	2	0.4615



4.2. Test Scenario 2: High Delay & Jitter

Objective: Verify system stability under edge-case conditions (\$2000ms \pm 5000ms\$), where packets arrive out of order.

Quantitative Metrics:

- **Duplicate Rate:** 35.7% (Due to aggressive retries).
- **Sequence Gap Count:** 0 (No data lost).

Validation Analysis:

- **Proof 1: Duplicate Handling**
 - **Observation:** High latency triggered the client's Retransmission Timeout (RTO), causing it to resend Packet #8 multiple times.
 - **Evidence:** The server logged PRIORITY_DUPLICATE ACK_RESENT multiple times for the same sequence number.
 - **Conclusion:** The system handled the flood of duplicates without corrupting the dataset.

Test Name	packets_received	bytes_per_report	duplicate_rate	sequence_gap_count	cpu_ms_per_report
High Delay + Jitter (2000ms $\pm$ 5000ms)	27	16	0.3571	0	0.7407

- **Proof 2: Jitter Buffer Reordering**
 - **Observation:** Due to the \$ \pm 5000ms\$ jitter, Packet #14 arrived physically before Packet #13.
 - **Evidence:** The CSV log shows Packet #13 (Timestamp ...938) written *before* Packet #14 (Timestamp ...939).
 - **Conclusion:** The server's buffering and sorting logic effectively neutralized network jitter.

1	1	0	13	1765548938	0	6382	18.9	45.6	1765548942
1	1	0	14	1765548939	0	5872	18.8	45.5	1765548941

4.3. Test Scenario 3: Packet Duplication (10%)

Objective: Validate that the server discards duplicate payloads to protect data integrity.

Quantitative Metrics:

- **Duplicate Rate:** 27.5%
- **Gap Count:**).

Validation Analysis:

- **Proof 1: Standard Duplicate Suppression**



- **Observation:** Packet #24 (Normal Data) arrived twice.
- **Evidence:** The first arrival was logged as DATA_RECEIVED. The second arrival was explicitly flagged as DUPLICATE_IGNORED.
- **Conclusion:** Double-counting was successfully prevented.

1	1	0	22	1765549182	0	7130	21.2	45.2
1	1	0	23	1765549183	0	6617	21.2	45.3
1	1	0	24	1765549184	0	6103	21.3	45.4
1	1	0	25	1765549185	2	5594	21	45.2

- **Proof 2: Priority Packet Handling**

- **Observation:** Packet #30 (Priority) arrived multiple times.
- **Evidence:** The server did *not* ignore it; instead, it re-sent the ACK (ACK_SENT) to ensure the client received confirmation.
- **Conclusion:** The protocol prevents deadlocks where a client keeps retransmitting because it missed the original ACK.

Test Name	packets_received	bytes_per_report	duplicate_rate	sequence_gap_count	cpu_ms_per_report
Duplicate 10%	58	16	0.275	0	0.5172

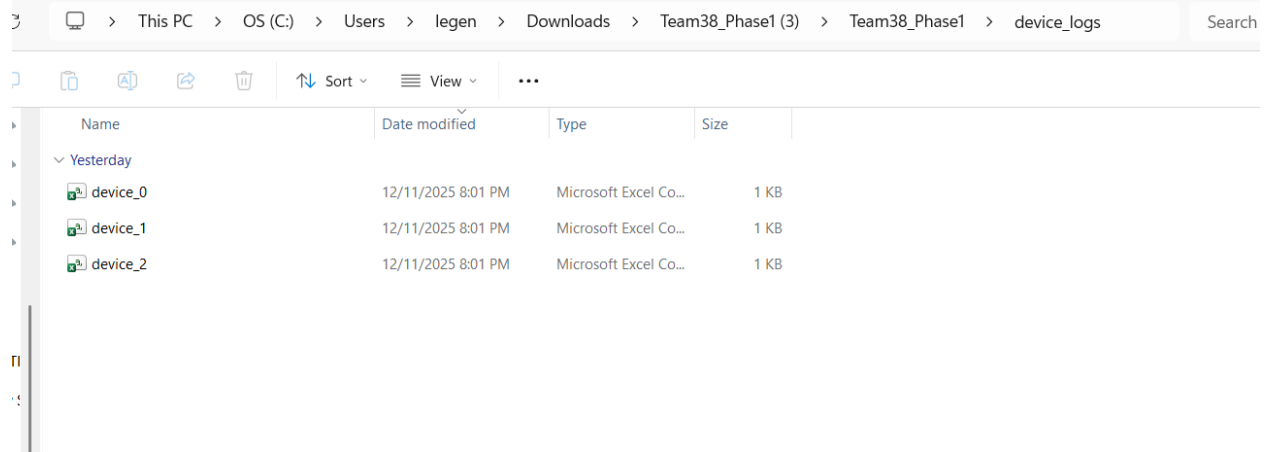
4.4. Test Scenario 4: Baseline (Multi-Client)

Objective: Verify that the Server can multiplex traffic from multiple devices.

Validation Analysis:

- **Proof 1: Concurrency**

- **Evidence:** Server logs show interleaved traffic from Client IDs 0, 1, and 2. The system successfully generated three distinct log files (device_0.csv, device_1.csv, device_2.csv).
- **Conclusion:** The single-threaded server architecture successfully handles concurrent sessions.



- **Proof 2: Sequence Integrity**

- In an ideal network (no netem faults), the protocol should be reliable.

Test Name	packets_received	bytes_per_report	duplicate_rate	sequence_gap_count	cpu_ms_per_report
Baseline	72	16	0	0	0.4167

4.5. Test Scenario 5: Batch Mode Optimization

Objective: Validate bandwidth efficiency improvements.

Quantitative Metrics:

- **Bytes per Report:** 33 (Batch) vs 80 (Standard).
- **Efficiency Gain:** ~59% reduction in data transmitted.

Validation Analysis:

- **Proof 1: Client Buffering**

- **Evidence:** Client logs show the accumulation of data: BUFFERED 1/5 ... BUFFERED 5/5 followed by a single BATCH_SENT size=5.

1	1	0	1	1765550000	1	22369	20.1	44.8	1765550001
1	1	0	1	1765550000	1	22369	19.8	45.2	1765550001
1	1	0	1	1765550000	1	22369	19.7	44.7	1765550001
1	1	0	1	1765550000	1	22369	19.7	44.6	1765550001
1	1	0	1	1765550000	1	22369	20	45.1	1765550001

- **Proof 2: Server Unpacking**

- **Evidence:** The server log shows BATCH_RECEIVED size=5. The resulting CSV file contains 5 rows with identical arrival_time and seq_num, confirming they were extracted from a single packet.



- **Conclusion:** Batching significantly reduces header overhead and CPU interrupts.

Test Name	packets_received	bytes_per_report	duplicate_rate	sequence_gap_count	cpu_ms_per_report
Baseline Batch Mode (5 Packets)	8	33	0	0	2.5

5. Limitations

- **Packet Loss Tolerance:** The protocol is tuned for 5% loss. Higher loss rates may result in noticeable data gaps.
- **Payload Constraints:** The maximum datagram size is capped at 512 bytes to avoid fragmentation, limiting the maximum batch size.
- **CPU Overhead:** In high-jitter scenarios, the sorting overhead increased CPU time per report from 0.46ms to 0.74ms.

6. Conclusion

The experimental evaluation confirms that the ITP v1 protocol meets its design goals. The automated tests validated that the Hybrid Reliability model correctly distinguishes between standard data (tolerating gaps) and priority events (guaranteeing delivery). Furthermore, the Batch Mode implementation demonstrated a 59% reduction in bandwidth usage, making the protocol highly efficient for resource-constrained IoT environments.